

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3067 -- Redes de Computadoras

SECCIÓN 20

Ing. Jorge Andrés Yass Coy



Proyecto 1

Uso de un protocolo existente (MCP)

Brandon Werner Rivera Cabrera 23088

GUATEMALA, 8 de agosto de 2026

0. Propuesta.

Se propone el desarrollo de un chatbot inteligente basado en un servidor MCP que integre a la empresa de venta de medicamentos con un asistente conversacional, como si se tratase de Soporte al cliente 24/7 sobre consulta de disponibilidad, precios y asesoría clínica limitada a ciertos productos. La información sale de una base de datos proporcionada por una empresa y el chatbot sería capaz de:

- Brindar información operativa en tiempo real como: stock, precios, ubicación de sucursales.
- Ofrecer orientación general sobre interacciones y uso de medicamentos, con escalamiento obligatorio a personal farmacéutico para casos que requieran juicio clínico.
- Servir tanto a clientes finales como a personal interno (farmacéuticos), con distintos niveles de acceso e información según el rol.

El objetivo es reducir tiempos de atención, mejorar la disponibilidad de información al cliente (24/7) y apoyar al personal interno en tareas de consulta rápida, sin sustituir el criterio profesional del farmacéutico en decisiones clínicas.

Los clientes enfrentan típicamente:

- Dudas repetitivas de bajo valor agregado, como precios y dosis, que consumen tiempo del personal.
- Riesgo de que los clientes busquen información de interacciones/dosis en fuentes no confiables de internet.

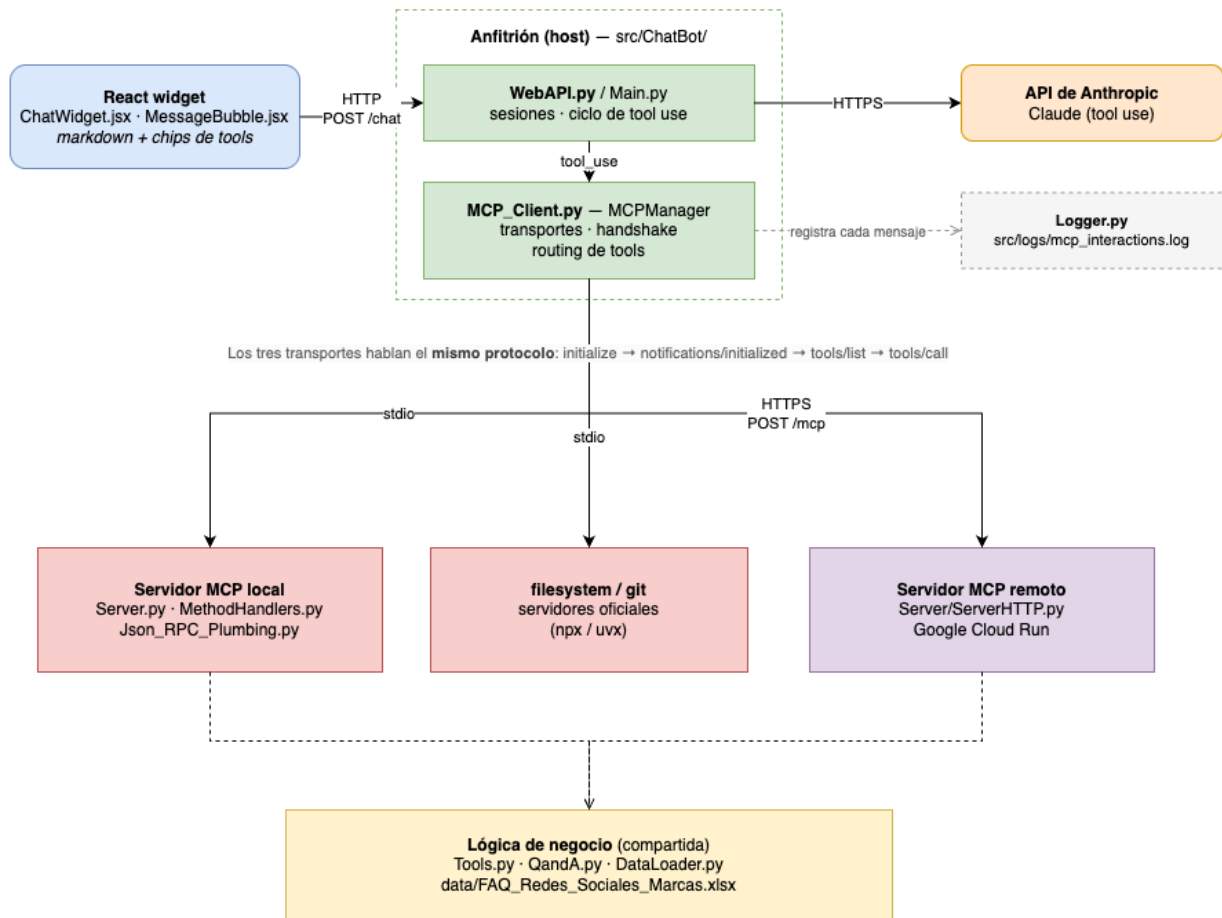
Un chatbot bien diseñado permite atender el primer tipo de consulta de forma automática y canalizar de forma segura el segundo tipo hacia fuentes verificadas y personal calificado.

1. Introducción

Los modelos de lenguaje de gran escala (LLM) presentan una limitación estructural: solo pueden responder a partir del conocimiento incorporado durante su entrenamiento y no disponen de ningún mecanismo propio para consultar sistemas externos ni actuar sobre ellos. Durante un tiempo, cada proveedor resolvió esta carencia con un formato propietario para describir e invocar herramientas, lo que obligaba a reescribir una misma integración para cada modelo. El Model Context Protocol (MCP), propuesto por Anthropic en noviembre de 2024, estandarizó ese intercambio: define cómo un servidor publica las herramientas que ofrece y cómo un anfitrión las descubre e invoca, de modo que el desarrollo de la herramienta queda desacoplado del modelo que la utiliza. MCP se apoya en JSON-RPC 2.0, un protocolo de llamada a procedimiento remoto que opera en la capa de aplicación de los modelos OSI y TCP/IP.

2. Arquitectura general

Arquitectura general — Chatbot con MCP (JSON-RPC 2.0 escrito a mano, sin SDK)



Componente	Archivo(s)	Responsabilidad
Anfitrión (host)	ChatBot/Main.py ChatBot/WebAPI.py	Conversación con el LLM, contexto de sesión, ciclo de tool use
Cliente MCP	ChatBot/MCP_Client.py	Transportes stdio y HTTP, handshake, routing de tools, log
Servidor MCP local	Server.py MethodHandlers.py Json_RPC_Plumbing.py	JSON-RPC sobre stdio
Servidor MCP remoto	Server/ServerHTTP.py	JSON-RPC sobre HTTP (Cloud Run)
Lógica de negocio	Tools.py QandA.py DataLoader.py	Catálogo de tools, búsqueda de FAQ en Excel
Interfaz web	ChatWidget.jsx MessageBubble.jsx	Widget de chat con markdown y chips de tools

Los colores corresponden a los del diagrama de arquitectura.

3. Especificación de los servidores MCP

Los servidores local y remoto comparten Tools.py, por lo que exponen exactamente el mismo catálogo. Solo difiere la capa de transporte.

3.1 Métodos JSON-RPC implementados

Método	Tipo	Descripción	Respuesta
initialize	Request	Negociación de versión y capacidades	protocolVersion, capabilities, serverInfo
notifications/initialized	Notification	El cliente confirma que terminó la inicialización.	Sin respuesta (HTTP 202 en remoto)
ping	Request	Verificación de conectividad	{}
tools/list	Request	Catálogo de herramientas disponibles	tools[] con name, description, inputSchema
tools/call	Request	Invocación de una herramienta	content[], isError

Versión de protocolo: 2025-06-18.

3.2 Códigos de error JSON-RPC

Código	Nombre	Cuándo se dispara
-32700	Parse error	El body no es JSON válido
-32600	Invalid request	Falta "jsonrpc": "2.0" o ruta/método HTTP incorrecto
-32601	Method not found	Método JSON-RPC desconocido, o tool inexistente en tools/call
-32602	Invalid params	Argumento obligatorio ausente o inválido
-32603	Internal error	Excepción no controlada al ejecutar la tool

3.3 Catálogo de tools

Tools de búsqueda aproximada (una por producto): get_faq_maxicrez, get_faq_diabelife, get_faq_colonditox, get_faq_geriaking_vital, get_faq_perenterol, get_faq_calmiderm

Parámetro	Tipo	Obligatorio	Descripción
Pregunta	string	Sí	Pregunta del usuario sobre el producto

Respuesta: lista JSON de hasta 5 candidatos {pregunta, respuesta, score} ordenados por similitud (token_set_ratio sobre texto normalizado), o mensaje de "no encontrado" con recomendación de consultar a un farmacéutico.

Tools de búsqueda exacta (una por producto): get_faq_maxicrez_exacta, get_faq_diabelife_exacta, get_faq_colonditox_exacta, get_faq_geriaking_vital_exacta, get_faq_perenterol_exacta, get_faq_calmiderm_exacta

Parámetro	Tipo	Obligatorio	Descripción
pregunta	string	Sí	Pregunta tal como aparece en la FAQ

Respuesta: objeto JSON {pregunta, respuesta} con la coincidencia exacta.

Tool de catálogo: consultar_faq_producto

Parámetro	Tipo	Obligatorio	Descripción
marca	string (enum)	Sí	Una de: maxicrez, diabelife, colonditox, geriaking vital, perenterol, calmiderm

Respuesta: Listado numerado de todas las preguntas disponibles para ese producto (sin respuestas).

3.4 Flujo de uso típico

Usuario: "¿Cuánto cuesta el Maxicrez?"

- LLM → tools/call consultar_faq_producto(marca="maxicrez")
 - Lista de 9 preguntas
- LLM identifica "¿Cuál es el precio del frasco?" como la relevante.
- LLM → tools/call get_faq_maxicrez_exacta(pregunta="¿Cuál es el precio del frasco?")
 - {pregunta, respuesta}
- LLM redacta la respuesta final al usuario.
 - Se redacta la respuesta según únicamente la información en la lista de preguntas, si es requerida más información, esta responde con "consulte a su médico"; adicionalmente, tiene prohibido inventar docis.

3.5 Endpoints del servidor remoto

Método HTTP	Ruta	Uso	Códigos de estado
POST	/mcp	Todos los mensajes JSON-RPC	200 (request), 202 (notificación), 400 (JSON inválido)
GET	/health	Health check de Cloud Run	200
*	otra ruta	—	404 con error -32600

Puerto: leído de la variable de entorno PORT (8080 en Cloud Run). Bind en 0.0.0.0.

3.6 Ejemplos de intercambio

- [2026-09-02T14:23:20] [farmacia-remota] --> {"jsonrpc": "2.0", "id": 1, "method": "initialize", "params": {"protocolVersion": "2025-06-18", "capabilities": {}, "clientInfo": {"name": "chatbot-host", "version": "1.0.0"}}}
- [2026-09-02T14:23:20] [farmacia-remota] <-- {"jsonrpc": "2.0", "id": 1, "result": {"protocolVersion": "2025-06-18", "capabilities": {"tools": {}}, "serverInfo": {"name": "farmacia-mcp-remote", "version": "1.0.0"}}}
- [2026-09-02T14:23:20] [farmacia-remota] --> {"jsonrpc": "2.0", "method": "notifications/initialized", "params": {}}
- [2026-09-02T14:23:21] [farmacia-remota] --> {"jsonrpc": "2.0", "id": 2, "method": "tools/list", "params": {}}
- [2026-09-02T14:23:21] [farmacia-remota] <-- {"jsonrpc": "2.0", "id": 2, "result": {"tools": [{"name": "get_faq_maxicrez", "description": "Consulta las preguntas frecuentes del producto. MaxicrezFormula el argumento 'pregunta' como una pregunta completa y explícita, expandiendo referencias vagas. Por ejemplo, si el usuario dice '¿cuánto cuesta?', envía '¿Cuál es el precio del producto?'.", "inputSchema": {"type": "object", "properties": {"pregunta": {"type": "string", "description": "Pregunta del usuario sobre el producto"}}, "required": ["pregunta"]}}, {"name": "get_faq_diabelife", "description": "Consulta las preguntas frecuentes del producto. DiabelifeFormula el argumento 'pregunta' como una pregunta completa y explícita, expandiendo referencias vagas. Por ejemplo, si el usuario dice '¿cuánto cuesta?', envía '¿Cuál es el precio del producto?'.", "inputSchema": {"type": "object", "properties": {"pregunta": {"type": "string", "description": "Pregunta del usuario sobre el producto"}}, "required": ["pregunta"]}}, {"name": "get_faq_colonditox", "description": "Consulta las preguntas frecuentes del producto. ColonditoxFormula el argumento 'pregunta' como una pregunta completa y explícita, expandiendo referencias vagas. Por ejemplo, si el usuario dice '¿cuánto cuesta?', envía '¿Cuál es el precio del producto?'.", "inputSchema": {"type": "object", "properties": {"pregunta": {"type": "string", "description": "Pregunta del usuario sobre el producto"}}, "required": ["pregunta"]}}]}
- [2026-09-04T12:49:15] [farmacia-remota] --> {"jsonrpc": "2.0", "id": 6, "method": "tools/call", "params": {"name": "get_faq_diabelife_exacta", "arguments": {"pregunta": "¿Qué es Diabelife?"}}}
- [2026-09-04T12:49:16] [farmacia-remota] <-- {"jsonrpc": "2.0", "id": 6, "result": {"content": [{"type": "text", "text": "\npregunta\n": "\n¿Qué es Diabelife?\n", "respuesta\n": "\nCon gusto, te comentamos que Diabelife es un suplemento de vitaminas, minerales y antioxidantes especialmente formulado para personas con diabetes. Su fórmula busca apoyar el bienestar general y satisfacer las necesidades nutricionales que suelen tener quienes viven con esta condición."}]}}

Es un complemento para la dieta diaria, ya que ayuda a cubrir deficiencias comunes sin impactar negativamente los niveles de azúcar. Si deseas leer y conocer un poco más de Diabelife puedes entrar a www.diabelife.club"

- `}]", "isError": false}}`

4. Análisis de la comunicación con Wireshark

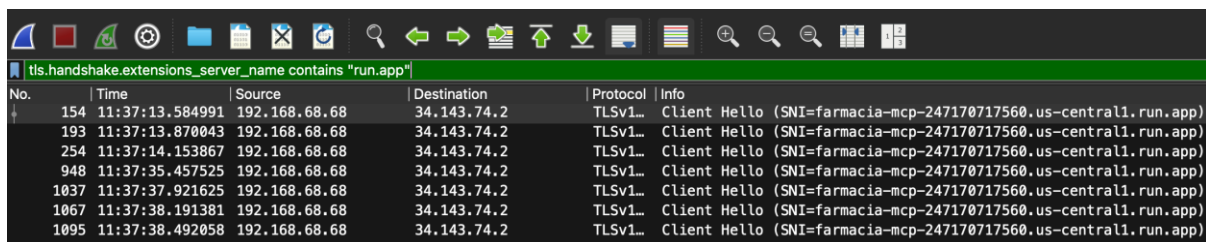
4.1 Metodología de captura

La captura se realizó sobre la interfaz Wi-Fi del equipo durante una sesión completa de uso del chatbot, siguiendo estos pasos:

1. Se inició la captura en Wireshark sobre la interfaz Wi-Fi, sin filtro de captura, para no descartar tráfico relevante.
2. Se levantó el contenedor de Docker con el chatbot y la interfaz web.
3. El anfitrión ejecutó la secuencia de inicialización contra el servidor MCP remoto.
4. Se mantuvo activo un health check automático cada 5 segundos contra el endpoint `/health`.
5. Se formuló una pregunta desde la interfaz web, lo que disparó el ciclo de invocación de herramientas.
6. Se detuvo la captura y se identificó la dirección del servidor mediante el nombre de servidor (SNI) presente en el saludo TLS.
7. Se aplicaron filtros de visualización sucesivos para aislar cada capa del modelo.

Identificación de la conexión. Dado que Cloud Run opera detrás de la infraestructura de balanceo de Google, resulta más confiable identificar la conversación por el SNI que por la dirección IP. El filtro utilizado fue:

`tls.handshake.extensions_server_name contains "run.app"`



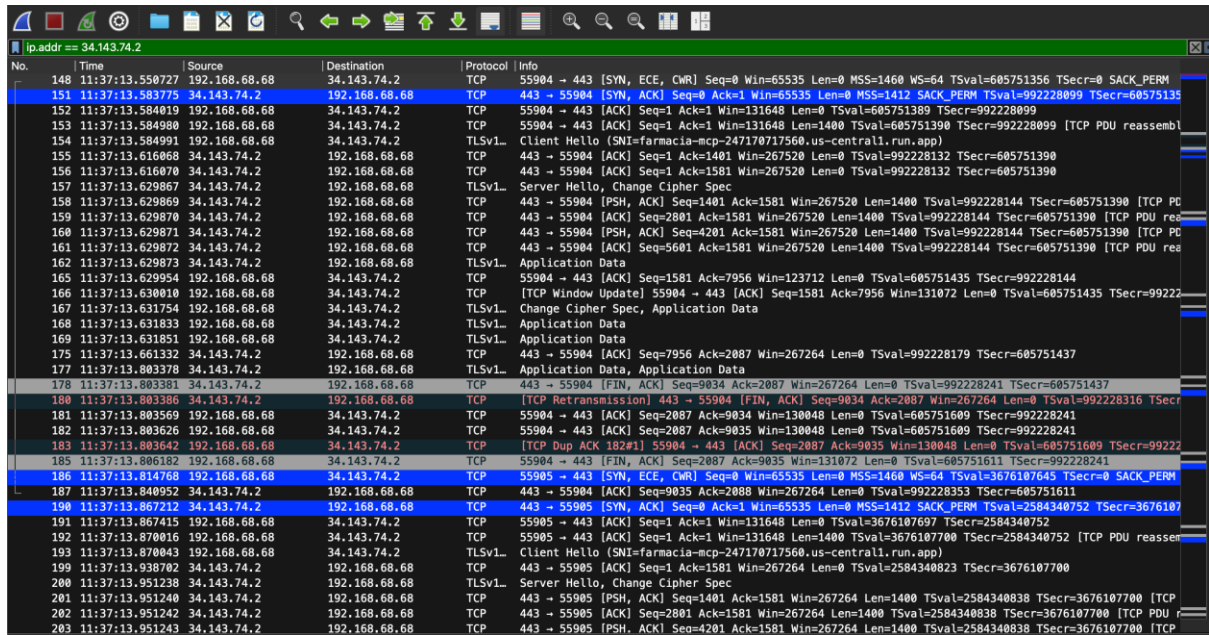
No.	Time	Source	Destination	Protocol	Info
154	11:37:13.584991	192.168.68.68	34.143.74.2	TLSv1...	Client Hello (SNI=farmacia-mcp-247170717560.us-central1.run.app)
193	11:37:13.870043	192.168.68.68	34.143.74.2	TLSv1...	Client Hello (SNI=farmacia-mcp-247170717560.us-central1.run.app)
254	11:37:14.153867	192.168.68.68	34.143.74.2	TLSv1...	Client Hello (SNI=farmacia-mcp-247170717560.us-central1.run.app)
948	11:37:35.457525	192.168.68.68	34.143.74.2	TLSv1...	Client Hello (SNI=farmacia-mcp-247170717560.us-central1.run.app)
1037	11:37:37.921625	192.168.68.68	34.143.74.2	TLSv1...	Client Hello (SNI=farmacia-mcp-247170717560.us-central1.run.app)
1067	11:37:38.191381	192.168.68.68	34.143.74.2	TLSv1...	Client Hello (SNI=farmacia-mcp-247170717560.us-central1.run.app)
1095	11:37:38.492058	192.168.68.68	34.143.74.2	TLSv1...	Client Hello (SNI=farmacia-mcp-247170717560.us-central1.run.app)

Este filtro reveló siete saludos TLS (Client Hello) dirigidos al servidor `farmacia-mcp-247170717560.us-central1.run.app`, todos desde `192.168.68.68` hacia `34.143.74.2`. A partir de ahí se seleccionó una conversación concreta con **Follow a TCP Stream**, quedando fijado el filtro base `tcp.stream eq 10` para el resto del análisis.

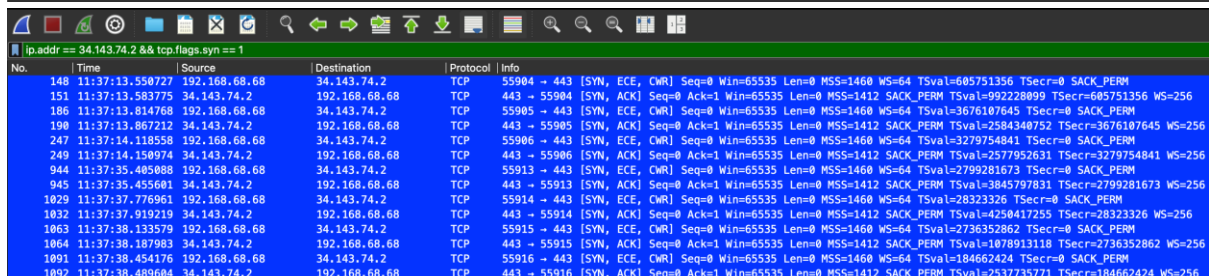
Parámetros de la conexión analizada:

Parámetro	Valor
-----------	-------

Cliente	192.168.68.68, puerto efimero 55004
Servidor	34.143.74.2, puerto 443
Nombre de servidor (SNI)	farmacia-mcp-247170717560.us-central1.run.app
Versión TLS	1.3
Flujo TCP	tcp.stream eq 10
Marca de tiempo inicial	11:37:13.550727



No.	Time	Source	Destination	Protocol	Info
148	11:37:13.550727	192.168.68.68	34.143.74.2	TCP	55984 → 443 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=605751356 TSecr=0 SACK_PERM
151	11:37:13.583775	34.143.74.2	192.168.68.68	TCP	443 → 55984 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM TSval=992228099 TSecr=605751356
152	11:37:13.584819	192.168.68.68	34.143.74.2	TCP	55984 → 443 [ACK] Seq=1 Ack=1 Win=131648 Len=0 TSval=605751389 TSecr=992228099
153	11:37:13.584901	192.168.68.68	34.143.74.2	TLSv1	Client Hello (SNI=farmacia-mcp-247170717560.us-central1.run.app)
154	11:37:13.584991	192.168.68.68	34.143.74.2	TLSv1	Server Hello, Change Cipher Spec
155	11:37:13.616868	34.143.74.2	192.168.68.68	TCP	443 → 55984 [PSH, ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228132 TSecr=605751390
156	11:37:13.616870	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1 Ack=1581 Win=267520 Len=0 TSval=992228132 TSecr=605751390
157	11:37:13.629867	34.143.74.2	192.168.68.68	TLSv1	Application Data
158	11:37:13.629869	34.143.74.2	192.168.68.68	TCP	443 → 55984 [PSH, ACK] Seq=2801 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390
159	11:37:13.629870	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=2801 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390
160	11:37:13.629871	34.143.74.2	192.168.68.68	TCP	443 → 55984 [PSH, ACK] Seq=4201 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390
161	11:37:13.629872	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=5601 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390
162	11:37:13.629873	34.143.74.2	192.168.68.68	TLSv1	Application Data
163	11:37:13.629954	192.168.68.68	34.143.74.2	TCP	55984 → 443 [ACK] Seq=1581 Ack=7956 Win=123712 Len=0 TSval=605751435 TSecr=992228144
166	11:37:13.630010	192.168.68.68	34.143.74.2	TCP	[TCP Window Update] 55984 → 443 [ACK] Seq=1581 Ack=7956 Win=131072 Len=0 TSval=605751435 TSecr=992228144
167	11:37:13.631754	192.168.68.68	34.143.74.2	TLSv1	Change Cipher Spec, Application Data
168	11:37:13.631833	192.168.68.68	34.143.74.2	TLSv1	Application Data
169	11:37:13.631851	192.168.68.68	34.143.74.2	TLSv1	Application Data
175	11:37:13.661332	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=7956 Ack=2087 Win=267264 Len=0 TSval=992228179 TSecr=605751437
177	11:37:13.803378	34.143.74.2	192.168.68.68	TLSv1	Application Data
178	11:37:13.803381	34.143.74.2	192.168.68.68	TCP	443 → 55984 [FIN, ACK] Seq=9034 Ack=2087 Win=267264 Len=0 TSval=992228241 TSecr=605751437
180	11:37:13.803388	34.143.74.2	192.168.68.68	TCP	[TCP Retransmission] 443 → 55984 [FIN, ACK] Seq=9034 Ack=2087 Win=267264 Len=0 TSval=992228316 TSecr=605751437
181	11:37:13.803569	192.168.68.68	34.143.74.2	TCP	55984 → 443 [ACK] Seq=2087 Ack=9034 Win=130048 Len=0 TSval=605751609 TSecr=992228241
182	11:37:13.803626	192.168.68.68	34.143.74.2	TCP	55984 → 443 [ACK] Seq=2087 Ack=9035 Win=130048 Len=0 TSval=605751609 TSecr=992228241
183	11:37:13.803642	192.168.68.68	34.143.74.2	TCP	[TCP Dup ACK 182#1] 55984 → 443 [ACK] Seq=2087 Ack=9035 Win=130048 Len=0 TSval=605751609 TSecr=992228241
185	11:37:13.806182	192.168.68.68	34.143.74.2	TCP	55984 → 443 [FIN, ACK] Seq=2087 Ack=9035 Win=131072 Len=0 TSval=605751611 TSecr=992228241
186	11:37:13.814768	192.168.68.68	34.143.74.2	TCP	55985 → 443 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=3676107645 TSecr=0 SACK_PERM
187	11:37:13.840952	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=9035 Ack=2088 Win=267264 Len=0 TSval=992228353 TSecr=605751611
190	11:37:13.867212	34.143.74.2	192.168.68.68	TCP	443 → 55985 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM TSval=2584340752 TSecr=3676107645
191	11:37:13.867415	192.168.68.68	34.143.74.2	TCP	55985 → 443 [ACK] Seq=1 Ack=1 Win=131648 Len=0 TSval=3676107697 TSecr=2584340752
192	11:37:13.870016	192.168.68.68	34.143.74.2	TCP	55985 → 443 [ACK] Seq=1 Ack=1 Win=131648 Len=1400 TSval=3676107700 TSecr=2584340752
193	11:37:13.870043	192.168.68.68	34.143.74.2	TLSv1	Client Hello (SNI=farmacia-mcp-247170717560.us-central1.run.app)
199	11:37:13.938702	34.143.74.2	192.168.68.68	TCP	443 → 55985 [ACK] Seq=1 Ack=1581 Win=267264 Len=0 TSval=2584340823 TSecr=3676107700
200	11:37:13.951238	34.143.74.2	192.168.68.68	TLSv1	Server Hello, Change Cipher Spec
201	11:37:13.951240	34.143.74.2	192.168.68.68	TCP	443 → 55985 [PSH, ACK] Seq=1401 Ack=1581 Win=267264 Len=1400 TSval=2584340838 TSecr=3676107700
202	11:37:13.951242	34.143.74.2	192.168.68.68	TCP	443 → 55985 [ACK] Seq=2801 Ack=1581 Win=267264 Len=1400 TSval=2584340838 TSecr=3676107700
203	11:37:13.951243	34.143.74.2	192.168.68.68	TCP	443 → 55985 [PSH, ACK] Seq=4201 Ack=1581 Win=267264 Len=1400 TSval=2584340838 TSecr=3676107700

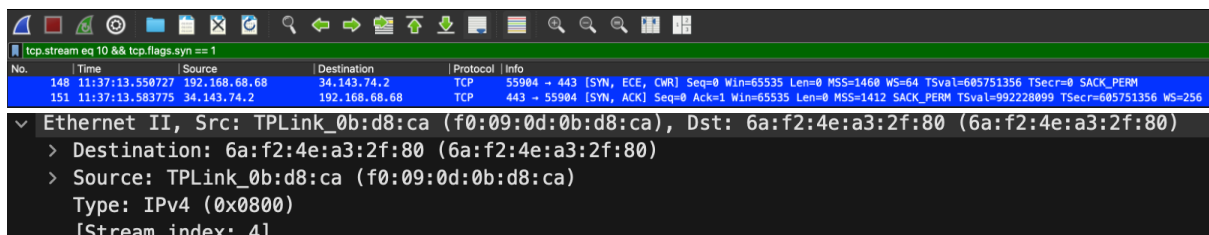


No.	Time	Source	Destination	Protocol	Info
148	11:37:13.550727	192.168.68.68	34.143.74.2	TCP	55984 → 443 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=605751356 TSecr=0 SACK_PERM
151	11:37:13.583775	34.143.74.2	192.168.68.68	TCP	443 → 55984 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM TSval=992228099 TSecr=605751356 WS=256
186	11:37:13.814768	192.168.68.68	34.143.74.2	TCP	55985 → 443 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=3676107645 TSecr=0 SACK_PERM
190	11:37:13.867212	34.143.74.2	192.168.68.68	TCP	443 → 55985 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM TSval=2584340752 TSecr=3676107645 WS=256
247	11:37:14.118558	192.168.68.68	34.143.74.2	TCP	55986 → 443 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=3279754841 TSecr=0 SACK_PERM
249	11:37:14.150974	34.143.74.2	192.168.68.68	TCP	443 → 55986 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM TSval=2577952631 TSecr=3279754841 WS=256
9913	11:37:35.405088	192.168.68.68	34.143.74.2	TCP	55913 → 443 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=2799281673 TSecr=0 SACK_PERM
945	11:37:35.455601	34.143.74.2	192.168.68.68	TCP	443 → 55913 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM TSval=3845797831 TSecr=2799281673 WS=256
1029	11:37:37.776961	192.168.68.68	34.143.74.2	TCP	55914 → 443 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=28323326 TSecr=0 SACK_PERM
1032	11:37:37.010219	34.143.74.2	192.168.68.68	TCP	443 → 55914 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM TSval=4258417255 TSecr=28323326 WS=256
1063	11:37:38.133579	192.168.68.68	34.143.74.2	TCP	55915 → 443 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=2736352862 TSecr=0 SACK_PERM
1064	11:37:38.187983	34.143.74.2	192.168.68.68	TCP	443 → 55915 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM TSval=1078913118 TSecr=2736352862 WS=256
1091	11:37:38.454176	192.168.68.68	34.143.74.2	TCP	55916 → 443 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=184662424 TSecr=0 SACK_PERM
1092	11:37:38.489604	34.143.74.2	192.168.68.68	TCP	443 → 55916 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM TSval=2537735771 TSecr=184662424 WS=256

4.2 Capa de enlace

Filtro aplicado: tcp.stream eq 10 && tcp.flags.syn == 1

En esta capa se observa el encapsulamiento Ethernet de las tramas que transportan el establecimiento de la conexión. Los paquetes 148 y 151 corresponden al SYN y al SYN-ACK respectivamente.



No.	Time	Source	Destination	Protocol	Info
148	11:37:13.550727	192.168.68.68	34.143.74.2	TCP	55984 → 443 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=605751356 TSecr=0 SACK_PERM
151	11:37:13.583775	34.143.74.2	192.168.68.68	TCP	443 → 55984 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM TSval=992228099 TSecr=605751356 WS=256

Ethernet II, Src: TPLink_0b:d8:ca (f0:09:0d:0b:d8:ca), Dst: 6a:f2:4e:a3:2f:80 (6a:f2:4e:a3:2f:80)	
>	Destination: 6a:f2:4e:a3:2f:80 (6a:f2:4e:a3:2f:80)
>	Source: TPLink_0b:d8:ca (f0:09:0d:0b:d8:ca)
Type: IPv4 (0x0800)	
[Stream index: 4]	

Un punto relevante de esta capa es que la dirección MAC de destino **no corresponde al servidor de Google**, sino al router de la red local. Las direcciones MAC solo tienen alcance dentro del dominio de difusión, por lo que se reescriben en cada salto; únicamente las direcciones IP se mantienen constantes de extremo a extremo. Este es el mecanismo que permite que el enlace opere de forma independiente del destino final del datagrama.

4.3 Capa de red

Filtro aplicado: tcp.stream eq 10 && ip

No.	Time	Source	Destination	Protocol	Info	(Protocol Length)
148	11:37:13.550727	192.168.68.68	34.143.74.2	TCP	55984 → 443 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=685751356 TSecr=0 SACK_PERM	
151	11:37:13.583775	34.143.74.2	192.168.68.68	TCP	443 → 55984 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM TSval=992228099 TSecr=685751356 WS=256	
152	11:37:13.584019	192.168.68.68	34.143.74.2	TCP	55984 → 443 [ACK] Seq=1 Ack=1 Win=131648 Len=0 TSval=685751389 TSecr=992228099	
153	11:37:13.584980	192.168.68.68	34.143.74.2	TCP	55984 → 443 [ACK] Seq=1 Ack=1 Win=131648 Len=1400 TSval=685751390 TSecr=992228099 [TCP PDU reassembled in 154]	14
154	11:37:13.584991	192.168.68.68	34.143.74.2	TLSv1..	Client Hello (SNI=farmacia-mcp-247170717560.us-central1.run.app)	2
155	11:37:13.616068	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1 Ack=1401 Win=267520 Len=0 TSval=992228132 TSecr=685751390	14
156	11:37:13.616070	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1 Ack=1581 Win=267520 Len=0 TSval=992228132 TSecr=685751390	14
157	11:37:13.629867	34.143.74.2	192.168.68.68	TLSv1..	Server Hello, Change Cipher Spec	14
158	11:37:13.629869	34.143.74.2	192.168.68.68	TCP	443 → 55984 [PSH, ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=685751390 [TCP PDU reassembled in 154]	14
159	11:37:13.629870	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=2801 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=685751390 [TCP PDU reassembled in 154]	14
160	11:37:13.629871	34.143.74.2	192.168.68.68	TCP	443 → 55984 [PSH, ACK] Seq=4201 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=685751390 [TCP PDU reassembled in 154]	14
161	11:37:13.629872	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=5601 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=685751390 [TCP PDU reassembled in 154]	14
162	11:37:13.629873	34.143.74.2	192.168.68.68	TLSv1..	Application Data	10
165	11:37:13.629954	192.168.68.68	34.143.74.2	TCP	55984 → 443 [ACK] Seq=1581 Ack=7956 Win=123712 Len=0 TSval=685751435 TSecr=992228144	
166	11:37:13.630010	192.168.68.68	34.143.74.2	TCP	[TCP Window Update] 55984 → 443 [ACK] Seq=1581 Ack=7956 Win=131072 Len=0 TSval=685751435 TSecr=992228144	
167	11:37:13.631754	192.168.68.68	34.143.74.2	TLSv1..	Change Cipher Spec, Application Data	1
168	11:37:13.631833	192.168.68.68	34.143.74.2	TLSv1..	Application Data	2
169	11:37:13.631851	192.168.68.68	34.143.74.2	TLSv1..	Application Data	2
175	11:37:13.661332	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=7956 Ack=2087 Win=267264 Len=0 TSval=992228179 TSecr=685751437	
177	11:37:13.803378	34.143.74.2	192.168.68.68	TLSv1..	Application Data, Application Data	11
178	11:37:13.803381	34.143.74.2	192.168.68.68	TCP	443 → 55984 [FIN, ACK] Seq=9034 Ack=2087 Win=267264 Len=0 TSval=992228241 TSecr=685751437	
180	11:37:13.803386	34.143.74.2	192.168.68.68	TCP	[TCP Retransmission] 443 → 55984 [FIN, ACK] Seq=9034 Ack=2087 Win=267264 Len=0 TSval=992228316 TSecr=685751437	
181	11:37:13.803569	192.168.68.68	34.143.74.2	TCP	55984 → 443 [ACK] Seq=2087 Ack=9034 Win=130048 Len=0 TSval=685751609 TSecr=992228241	
182	11:37:13.803626	192.168.68.68	34.143.74.2	TCP	55984 → 443 [ACK] Seq=2087 Ack=9035 Win=130048 Len=0 TSval=685751609 TSecr=992228241	
183	11:37:13.803642	192.168.68.68	34.143.74.2	TCP	[TCP Dup ACK 182x1] 55984 → 443 [ACK] Seq=2087 Ack=9035 Win=130048 Len=0 TSval=685751609 TSecr=992228241	
185	11:37:13.806182	192.168.68.68	34.143.74.2	TCP	55984 → 443 [FIN, ACK] Seq=2087 Ack=9035 Win=131072 Len=0 TSval=685751611 TSecr=992228241	
187	11:37:13.840952	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=9035 Ack=2088 Win=267264 Len=0 TSval=992228353 TSecr=685751611	

Toda la comunicación se realizó sobre IPv4. Los datagramas capturados presentan:

	Valor
Dirección origen	192.168.68.68 (dirección privada, RFC 1918)
Dirección destino	34.143.74.2 (dirección pública, rango de Google Cloud)
Protocolo	6 (TCP)
Versión	IPv4

La dirección de origen es privada, lo que implica que el enrutador doméstico aplica traducción de direcciones (NAT) antes de enviar el tráfico hacia Internet. Desde la perspectiva del servidor de Cloud Run, el origen aparente es la dirección pública asignada por el proveedor de servicio, no 192.168.68.68.

La dirección de destino, 34.143.74.2, pertenece al rango de Google Cloud. Cabe notar que esta dirección corresponde al balanceador de carga que atiende el servicio, no a la instancia del contenedor: Cloud Run abstrae por completo la ubicación real de la ejecución.

4.4 Capa de transporte

Filtro aplicado: tcp.stream eq 10 && (tcp.flags.syn == 1 || tcp.flags.fin == 1)

The first image shows the initial SYN exchange:

No.	Time	Source	Destination	Protocol	Info
148	11:37:13.550727	192.168.68.68	34.143.74.2	TCP	55984 → 443 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=605751356 TSecr=0 SACK_PERM
151	11:37:13.583775	34.143.74.2	192.168.68.68	TCP	443 → 55984 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM TSval=992228099 TSecr=605751356 WS=256
152	11:37:13.584019	192.168.68.68	34.143.74.2	TCP	443 → 55984 [FIN, ACK] Seq=9034 Ack=2087 Win=267264 Len=0 TSval=992228241 TSecr=605751437

The second image shows the continuation of the connection:

No.	Time	Source	Destination	Protocol	Info
155	11:37:13.616668	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1 Ack=1 Win=131648 Len=0 TSval=605751389 TSecr=992228099
156	11:37:13.616070	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1 Ack=1 Win=131648 Len=0 TSval=605751389 TSecr=992228099
157	11:37:13.629867	34.143.74.2	192.168.68.68	TLSv1	Client Hello [SNI=farmacis-mcp-247170717560.us-central1.run.app]
158	11:37:13.629869	34.143.74.2	192.168.68.68	TLSv1	Server Hello, Change Cipher Spec
159	11:37:13.629870	34.143.74.2	192.168.68.68	TCP	443 → 55984 [PSH, ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
160	11:37:13.629871	34.143.74.2	192.168.68.68	TCP	443 → 55984 [PSH, ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
161	11:37:13.629872	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
162	11:37:13.629873	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
163	11:37:13.629874	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
164	11:37:13.629875	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
165	11:37:13.629876	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
166	11:37:13.629877	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
167	11:37:13.629878	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
168	11:37:13.629879	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
169	11:37:13.629880	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
170	11:37:13.629881	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
171	11:37:13.629882	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
172	11:37:13.629883	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
173	11:37:13.629884	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
174	11:37:13.629885	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
175	11:37:13.629886	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
176	11:37:13.629887	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
177	11:37:13.629888	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
178	11:37:13.629889	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
179	11:37:13.629890	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
180	11:37:13.629891	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
181	11:37:13.629892	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
182	11:37:13.629893	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
183	11:37:13.629894	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
184	11:37:13.629895	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
185	11:37:13.629896	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
186	11:37:13.629897	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]
187	11:37:13.629898	34.143.74.2	192.168.68.68	TCP	443 → 55984 [ACK] Seq=1401 Ack=1581 Win=267520 Len=1400 TSval=992228144 TSecr=605751390 [TCP PDU reassembled in 16..]

Establecimiento de la conexión

El saludo de tres vías se observa en los paquetes 148, 151 y 152:

#	Tiempo	Origen a Destino	Bandera	Parámetros negociados
148	11:37:13.550727	192.168.68.68:55004 a 34.143.74.2:443	SYN	Seq=0, Win=65535, MSS=1460, WS=64, SACK PERM
151	11:37:13.583775	34.143.74.2:443 a 192.168.68.68:55004	SYN, ACK	Seq=0, Ack=1, Win=65535, MSS=1412, WS=256, SACK PERM
152	11:37:13.584019	192.168.68.68:55004 a 34.143.74.2:443	ACK	Seq=1, Ack=1, Win=131648

Observaciones sobre la negociación:

- **MSS asimétrico.** El cliente anuncia un tamaño máximo de segmento de 1460 bytes, mientras que el servidor anuncia 1412. Esta diferencia se explica por el encapsulamiento adicional que utiliza la infraestructura de Google internamente, y obliga a que los segmentos en esa dirección sean más pequeños.
- **Escalado de ventana.** Ambos extremos negocian el factor de escala, lo que permite ventanas de recepción superiores al límite de 65535 bytes del campo original. Tras el saludo, la ventana efectiva del cliente asciende a 131648 bytes.
- El tiempo entre el SYN y el SYN-ACK es de aproximadamente **33 ms**, lo que corresponde a la latencia de ida y vuelta hasta el centro de datos de us-central1.

Transferencia y cierre

Filtro aplicado: tcp.stream eq 10

Durante la fase de transferencia se observan segmentos con la bandera PSH, indicando que los datos deben entregarse inmediatamente a la capa de aplicación, así como la marca TCP

PDU reassembled in que revela que algunos mensajes TLS exceden el MSS y se reparten entre varios segmentos.

La conexión se cierra a partir del paquete 178, iniciado por el servidor. En la captura se aprecian además fenómenos de control de flujo y recuperación:

- **Paquete 180:** retransmisión TCP del segmento FIN, indicando que el acuse correspondiente no llegó dentro del tiempo esperado.
- **Paquete 183:** acuse duplicado (TCP Dup ACK), señal de que se recibió un segmento fuera de orden o repetido.
- **Paquete 166:** actualización de ventana (TCP Window Update), mediante la cual el receptor informa que dispone de más espacio de recepción.

Estos eventos son parte del funcionamiento normal del protocolo y evidencian los mecanismos de fiabilidad y control de flujo de TCP operando sobre una red real.

Ausencia de reutilización de conexiones

Un hallazgo relevante del análisis es que el filtro por SNI reveló **siete saludos TLS distintos** en un intervalo de aproximadamente 25 segundos, cada uno con su propio flujo TCP:

# paquete	Tiempo	Intervalo respecto al anterior
154	11:37:13.584991	00 s
193	11:37:13.870043	0.285 s
254	11:37:14.153867	0.284 s
948	11:37:35.457525	21.30 s
1037	11:37:37.921625	2.46 s
1067	11:37:38.191381	0.270 s
1095	11:37:38.492058	0.301 s

Aunque el servidor remoto declara HTTP/1.1 y por tanto admite conexiones persistentes, el cliente MCP implementado utiliza `urllib.request.urlopen` para cada mensaje JSON-RPC, lo que abre una conexión TCP y una sesión TLS nuevas en cada invocación. El costo es apreciable: cada mensaje requiere un saludo TCP de tres vías más una negociación TLS completa antes de transmitir un solo byte útil.

4.5 Capa de aplicación

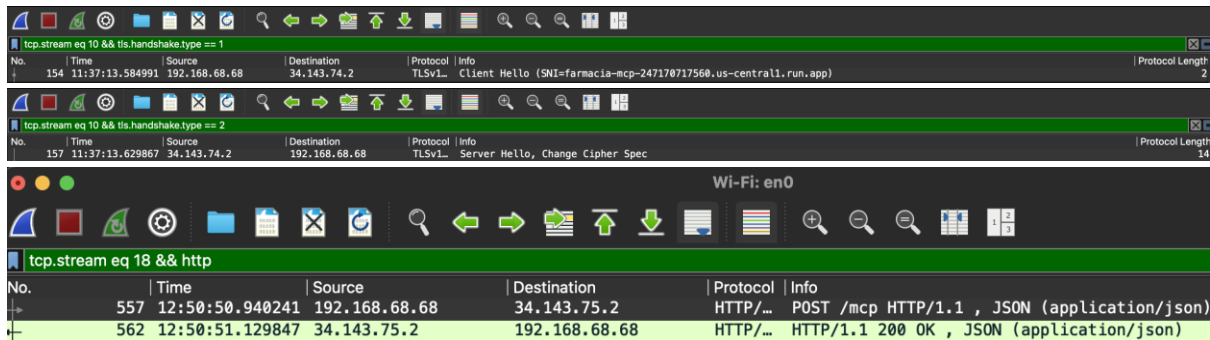
Negociación TLS

Filtros aplicados: `tcp.stream eq 10 && tls.handshake.type == 1` y `tcp.stream eq 10 && tls.handshake.type == 2`

#	Tiempo	Mensaje	Contenido relevante
---	--------	---------	---------------------

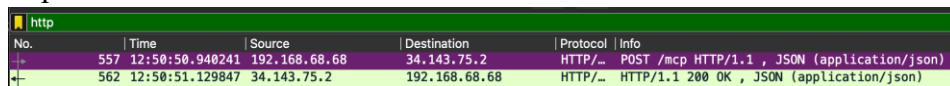
154	11:37:13.584991	Client Hello	SNI = farmacia-mcp-247170717560.us-central1.run.app
157	11:37:13.629867	Server Hello, Change Cipher Spec	Selección de parámetros criptográficos

El campo **SNI** del saludo del cliente es el único elemento de la negociación que viaja en claro y que identifica el destino lógico de la petición. Esto es necesario porque una misma dirección IP atiende múltiples servicios, y el servidor debe saber qué certificado presentar antes de que exista un canal cifrado.



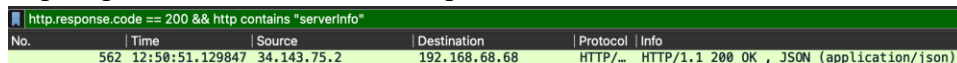
No.	Time	Source	Destination	Protocol	Info
154	11:37:13.584991	192.168.68.68	34.143.74.2	TLSv1	Client Hello (SNI=farmacia-mcp-247170717560.us-central1.run.app)
157	11:37:13.629867	34.143.74.2	192.168.68.68	TLSv1	Server Hello, Change Cipher Spec
557	12:50:50.940241	192.168.68.68	34.143.75.2	HTTP	POST /mcp HTTP/1.1, JSON (application/json)
562	12:50:51.129847	34.143.75.2	192.168.68.68	HTTP	HTTP/1.1 200 OK, JSON (application/json)

- initialize (petición)
 - http contains "\"method\": \"initialize\""



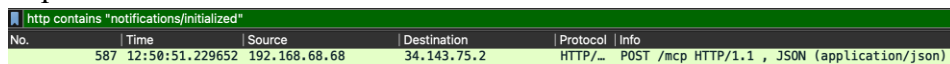
No.	Time	Source	Destination	Protocol	Info
557	12:50:50.940241	192.168.68.68	34.143.75.2	HTTP	POST /mcp HTTP/1.1, JSON (application/json)
562	12:50:51.129847	34.143.75.2	192.168.68.68	HTTP	HTTP/1.1 200 OK, JSON (application/json)

- respuesta a initialize
 - http.response.code == 200 && http contains "serverInfo"



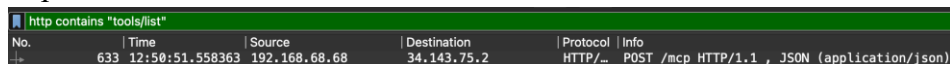
No.	Time	Source	Destination	Protocol	Info
562	12:50:51.129847	34.143.75.2	192.168.68.68	HTTP	HTTP/1.1 200 OK, JSON (application/json)

- Notifications/initialized
 - http contains "notifications/initialized"



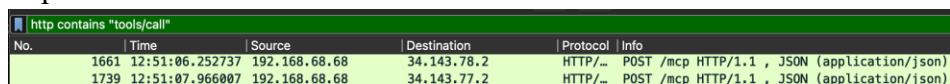
No.	Time	Source	Destination	Protocol	Info
587	12:50:51.229652	192.168.68.68	34.143.75.2	HTTP	POST /mcp HTTP/1.1, JSON (application/json)

- Tools/list
 - http contains "tools/list"



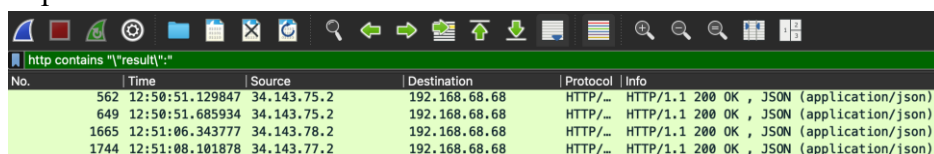
No.	Time	Source	Destination	Protocol	Info
633	12:50:51.558363	192.168.68.68	34.143.75.2	HTTP	POST /mcp HTTP/1.1, JSON (application/json)

- Tools/call
 - http contains "tools/call"



No.	Time	Source	Destination	Protocol	Info
1661	12:51:06.252737	192.168.68.68	34.143.78.2	HTTP	POST /mcp HTTP/1.1, JSON (application/json)
1739	12:51:07.966007	192.168.68.68	34.143.77.2	HTTP	POST /mcp HTTP/1.1, JSON (application/json)

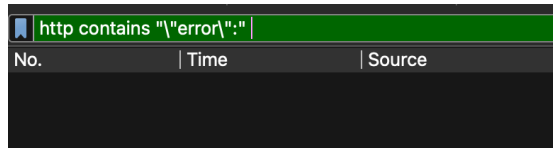
- Respuestas exitosas
 - http contains "\"result\":"



No.	Time	Source	Destination	Protocol	Info
562	12:50:51.129847	34.143.75.2	192.168.68.68	HTTP	HTTP/1.1 200 OK, JSON (application/json)
649	12:50:51.685934	34.143.75.2	192.168.68.68	HTTP	HTTP/1.1 200 OK, JSON (application/json)
1665	12:51:06.343777	34.143.78.2	192.168.68.68	HTTP	HTTP/1.1 200 OK, JSON (application/json)

- Respuestas de error

- http contains "\"error\":"



-

Correspondencia entre semántica JSON-RPC y códigos HTTP

El transporte HTTP obliga a resolver una diferencia estructural entre ambos protocolos: JSON-RPC contempla mensajes que no reciben respuesta (las notificaciones, identificadas por la ausencia del campo id), mientras que en HTTP toda petición debe recibir un código de estado. La implementación resuelve este mapeo de la siguiente forma:

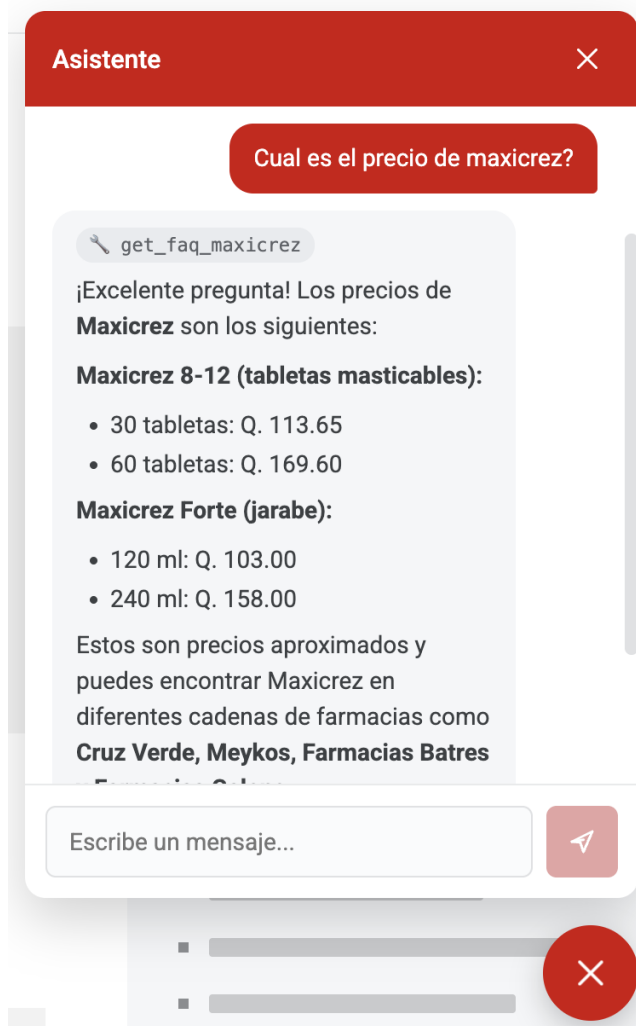
Situación JSON-RPC	Respuesta HTTP	Justificación
Petición atendida (con id)	200 OK + cuerpo JSON-RPC	Respuesta ordinaria
Notificación (sin id)	202 Accepted, sin cuerpo	El mensaje se aceptó pero no genera respuesta
Cuerpo con JSON malformado	400 Bad Request + error - 32700	Error de análisis sintáctico
Ruta distinta de /mcp	404 Not Found + error - 32600	Petición inválida a nivel de transporte

5. Funcionamiento del chatbot

- **Contexto.**
 - WebAPI.py guarda un diccionario sesiones: dict[str, list] que mapea session_id (lista de mensajes). La API de Anthropic no guarda estado, así que en cada messages.create() se manda el historial completo de esa sesión. Dos detalles: el turno del asistente se guarda tal cual lo devolvió la API (respuesta.content, la lista de bloques, no solo el texto), porque los bloques tool_use tienen que seguir ahí para que la siguiente llamada valide; y los resultados de las tools se agregan como mensaje con role: "user", que es la convención de la API. Si la llamada al LLM falla, se hace rollback (messages[:-1]) para no dejar el historial corrupto. Todo vive en memoria: Reiniciar el proceso borra las conversaciones, y DELETE /sesion/{id} borra una.
- **Ciclo de tool use.**
 - Es un while True:
 - Se llama al LLM con el historial y tools=manager.anthropic_tools(), donde se traduce inputSchema (MCP) a input_schema (Anthropic).
 - Se guarda la respuesta en el historial.
 - Si stop_reason != "tool_use", se juntan los bloques de texto y se devuelve. Fin.

- Si no, por cada bloque `tool_use` se llama a `manager.call_tool(nombre, input)`, que busca en `tool_routing` a qué servidor pertenece esa tool y le manda un `tools/call` por JSON-RPC. Cada resultado se envuelve en `{"type": "tool_result", "tool_use_id": ...}`.
 - Los resultados se mandan de vuelta como un solo mensaje `user` y se repite el ciclo: el modelo ya ve las respuestas y decide si contesta o pide más tools.
 - Los errores del servidor no lanzan excepción: `call_tool` los devuelve como texto (`[server error -32602] ...`), así el modelo puede reaccionar en vez de tumbar la conversación.
- **Registro.**
 - `log_mcp(servidor, dirección, payload)` escribe una línea con timestamp ISO, nombre del servidor, `-->` o `<--` y el JSON completo del mensaje. Se llama desde `MCP_Client.py` en `request()` y `notify()`, antes de enviar y al recibir, así que queda el par completo de ida y vuelta. Escribe a `stderr` y además hace `append` a `src/logs/mcp_interactions.log`. `read_log(n)` devuelve las últimas `n` líneas: es lo que sirve el endpoint `GET /log` y el comando `/log` de la consola. Solo se registra tráfico MCP, no los turnos con el LLM.

Demo:

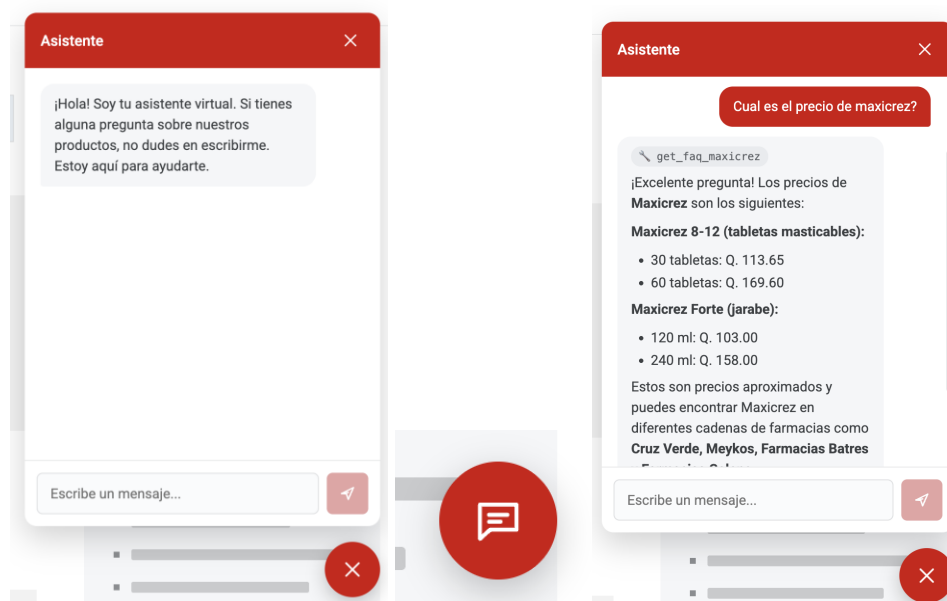


- [2026-09-07T17:21:11] [farmacia-remota] --> {"jsonrpc": "2.0", "id": 3, "method": "tools/call", "params": {"name": "get_faq_maxicrez", "arguments": {"pregunta": "¿Cuál es el precio de Maxicrez?"}}}
- [2026-09-07T17:21:11] [farmacia-remota] <-- {"jsonrpc": "2.0", "id": 3, "result": {"content": [{"type": "text", "text": "[
 - {"pregunta": "¿Precio?", "respuesta": "Con gusto, el precio aproximado de MaxicreZ 8-12 de 30 tabletas es de Q. 113.65 y el de 60 tabletas es de Q.169.60. Lo encuentras en las diferentes cadenas de farmacias como Cruz Verde, Meykos, Farmacias Batres y Farmacias Galeno.", "score": 100},
 - {"pregunta": "¿Qué es MaxicreZ?", "respuesta": "MaxicreZ es una familia de suplementos para niños cuya base principal es el Zinc, para fomentar el crecimiento en los peques. Contamos con Maxicrez Clásico y Forte en presentación de jarabe sabor banano. Ayudan al sistema inmunológico a estar en óptimas condiciones y promueve la formación de defensas en el cuerpo, también aumenta el apetito en los niños y los ayuda en su crecimiento debido a las vitaminas que contiene. MaxicreZ 8-12 es Zinc con vitaminas y minerales, es ideal para niños y adolescentes que están en etapa de desarrollo, ayuda a fortalecer las defensas del cuerpo y evitar
"]}]}

- enfermedades. Estas son tabletas masticables con delicioso sabor a chicle. \", \\\"score\\\": 85},
- {\\\"pregunta\\\": \\\"¿Cuál es el precio del frasco?\\\", \\\"respuesta\\\": \\\"Con gusto, el precio aproximado de Maxicrez 8-12 de 30 tabletas es de Q. 113.65 y el de 60 tabletas es de Q.169.60. Lo encuentras en las diferentes cadenas de farmacias como Cruz Verde, Meykos, Farmacias Batres y Farmacias Galeno.\\\", \\\"score\\\": 81},
 - {\\\"pregunta\\\": \\\"¿Cuáles son las presentaciones de Maxicrez Forte?\\\", \\\"respuesta\\\": \\\"Actualmente, contamos con 2 presentaciones, de 120 ml y de 240 ml. El precio de 120 ml es de Q. 103.00 y el precio de 240 ml es de Q158.00 (aproximadamente). \\\", \\\"score\\\": 71},
 - {\\\"pregunta\\\": \\\"¿Cuáles son las presentaciones de Maxicrez 8-12?\\\", \\\"respuesta\\\": \\\"Actualmente, contamos con 2 presentaciones. De 30 tabletas y de 60 tabletas. El precio de 30 tabletas es de Q.113.65.00 y el precio de 60 tabletas es de Q169.60 aproximadamente. \\\", \\\"score\\\": 69}}]
-], \"isError\": false}}

6. Interfaz de usuario (extra)

Decisiones de HCI: color corporativo, chips de tools como transparencia, indicador de carga, accesibilidad por teclado, mensajes de error visibles, markdown en respuestas.



7. Dificultades y cómo se resolvieron

Durante el desarrollo del proyecto se presentaron dificultades en tres frentes: la implementación manual del protocolo, la calidad de la búsqueda de preguntas frecuentes y el

despliegue del servidor remoto. A continuación se describen las más relevantes, en el orden en que aparecieron.

7.1 Ausencia de referencia para la implementación manual

El ejemplo oficial del servidor de clima (weather server) que se tomó como punto de partida está construido sobre el SDK de MCP, que abstrae por completo la capa JSON-RPC mediante decoradores. Al buscar en ese ejemplo la implementación del protocolo, no existía: estaba oculta dentro de la librería. Esto obligó a trabajar directamente con la especificación de MCP y de JSON-RPC 2.0 para entender el formato exacto de cada mensaje, el ciclo de vida y la diferencia entre peticiones, respuestas y notificaciones. Aunque implicó más tiempo inicial, fue precisamente lo que permitió comprender el protocolo en lugar de solo consumirlo.

7.2 Contaminación del canal stdout en el transporte stdio

En el transporte stdio, la salida estándar es el canal exclusivo del protocolo: cada línea debe ser un mensaje JSON-RPC válido. Durante las primeras pruebas, un `print()` de depuración que informaba cuántas preguntas se habían cargado por producto (Maxicrez: 9 preguntas) se escribía en stdout, y el cliente fallaba al intentar parsearlo como JSON. La solución fue redirigir todo el logging del servidor a stderr mediante una función `log()` centralizada, y hacer al cliente tolerante a líneas que no sean JSON para facilitar el diagnóstico. Esta distinción entre canal de datos y canal de diagnóstico resultó ser un principio recurrente en el resto del proyecto.

7.4 Desalineación entre tools/list y tools/call

Al experimentar con dos diseños de herramientas (una tool por producto frente a una tool genérica con el producto como parámetro), en un punto el servidor anunciaba herramientas en `tools/list` que `call_tool` no sabía manejar, y viceversa. El síntoma era un error `Unknown tool` para herramientas que sí aparecían en el catálogo. Se resolvió definiendo el catálogo y el enrutamiento a partir de una única estructura de datos (MARCAS), de modo que agregar un producto implica modificar un solo lugar y ambas listas no pueden divergir.

7.5 Búsqueda de preguntas frecuentes: tildes, mayúsculas y sinónimos

La búsqueda de la pregunta más parecida en el archivo Excel presentó dos problemas distintos. El primero, superficial: diferencias de mayúsculas, tildes y signos de interrogación reducían el puntaje de similitud aunque el texto fuera esencialmente el mismo. Se resolvió con una función de normalización (minúsculas, eliminación de diacríticos y puntuación) aplicada a ambos lados de la comparación, conservando el texto original para mostrarlo al usuario.

El segundo problema es de fondo: la comparación léxica no reconoce sinónimos. "¿Cuánto cuesta?" y "¿Cuál es el precio del frasco?" no comparten casi ninguna palabra, por lo que ningún algoritmo de similitud de cadenas las relaciona. En lugar de incorporar modelos de embeddings (que habrían complicado el despliegue), se optó por una división de responsabilidades: el servidor realiza una recuperación léxica amplia (top-N candidatos con `token_set_ratio` y umbral bajo) y expone además una herramienta que lista todas las preguntas disponibles; el modelo de lenguaje, que sí entiende semántica, selecciona la pregunta correcta. Esta decisión también dejó al descubierto un error de importación en el que el editor había resuelto `process` hacia `concurrent.futures` en lugar de `rapidfuzz`, corregido con un alias explícito.

8. Conclusiones

- **La implementación manual del protocolo expone decisiones de diseño que un SDK oculta.** Construir la capa JSON-RPC a mano obligó a resolver explícitamente cuestiones que una librería resuelve de forma invisible: qué mensajes llevan identificador y cuáles no, qué código de error corresponde a cada tipo de falla, cómo se delimita un mensaje sobre un flujo continuo de bytes, y en qué orden deben ocurrir los intercambios de inicialización.
- **MCP es genuinamente independiente del transporte.** El mismo catálogo de herramientas y la misma lógica de negocio se sirvieron por entrada/salida estándar de forma local y por HTTP sobre TLS desde Cloud Run, compartiendo el archivo de lógica de negocio sin modificación alguna. Desde la perspectiva del anfitrión, ambos servidores son intercambiables: el cliente implementa dos clases con idéntica interfaz y el resto del sistema no distingue cuál está en uso.
- **El transporte impone traducciones semánticas que el protocolo no anticipa.** JSON-RPC contempla mensajes que no reciben respuesta, mientras que HTTP exige un código de estado para toda petición.
- **Distribuir la responsabilidad entre el servidor y el modelo fue más efectivo que perfeccionar el algoritmo de búsqueda.** La comparación léxica de cadenas resulta insuficiente para relacionar preguntas semánticamente equivalentes que no comparten vocabulario. En lugar de incorporar modelos de embeddings se optó por que el servidor realizara una recuperación amplia y por que el modelo de lenguaje, que sí interpreta significado, seleccionara la respuesta pertinente.

9. Referencias

- JSON-RPC 2.0 Specification — <https://www.jsonrpc.org/specification>
- MCP Architecture — <https://modelcontextprotocol.io/docs/learn/architecture>

- MCP Specification 2025-06-18 — <https://modelcontextprotocol.io/specification/2025-06-18>
- MCP Servers (filesystem, git) — <https://github.com/modelcontextprotocol/servers>
- Anthropic API — Tool use — <https://docs.claude.com/en/docs/build-with-claude/tool-use>
- Google Cloud Run — <https://cloud.google.com/run/docs>